

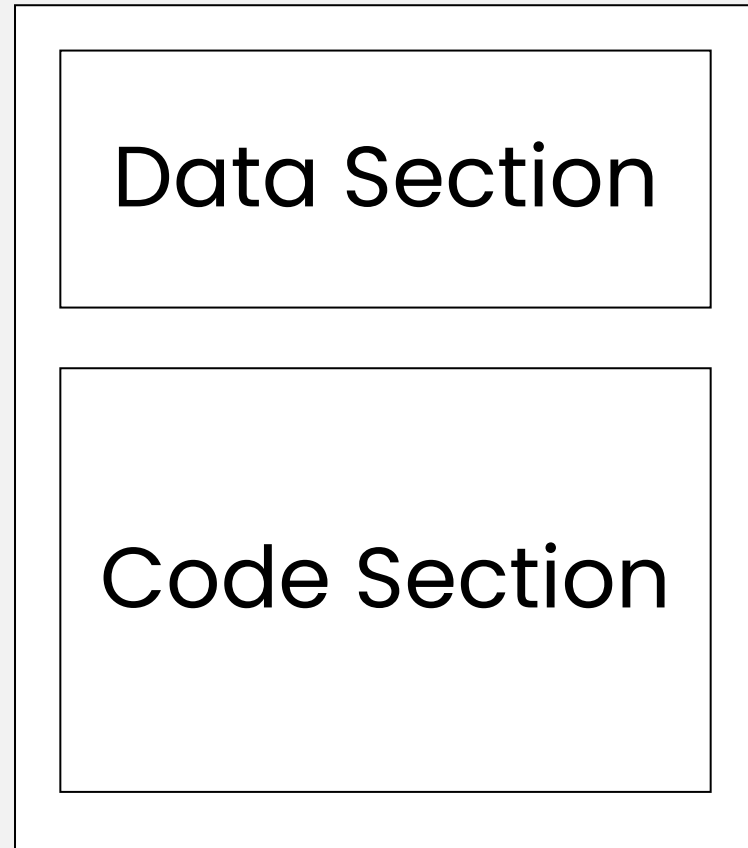
RISC-V

Introduction

gem5 and RISC-V

Ernesto Sanchez

ASSEMBLER PROGRAMS



Assembler program

Data Section

Variables
Constants

Code Section

Program
Routines
Subroutines

Data Section

```
***** RISC-V INITIAL PROGRAM*****  
#-----  
# AddVectorValues.s  
# this program adds the element in a vector  
# requires integer data in mem  
#-----
```

← Program Title

.data

← Assembler Directives

V1: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

← Constants

result: .word 0

← Variables

Code Section

	.Code		
	.global	main	
main:	addi	r1,r0,Info	*** Read value from stdin
	Jal	Input	into R1
	movi2fp	f10,r1	*** init values
	cvti2d	f0,f10	#R1 -> D0 D0..Count
	addi	r2,r0,1	register
	movi2fp	f11,r2	#1 -> D2 D2..result
	cvti2d	f2,f11	
	Movd	f4,f2	#1-> D4 D4..Constant 1
Loop:	led	f0,f4	*** Break loop if D0 = 1
	bfpt	EndL	#D0<=1 ?
	Multd	f2,f2,f0	*** Multiplication and
	subd	f0,f0,f4	next loop
	j	Loop	*** write result to tdout
EndL:	sd	Print,f2	
	addi	r14,r0,Print	
	trap	5	*** end

■ Assembler Directives

■ Labels

■ OPcode

■ Operators

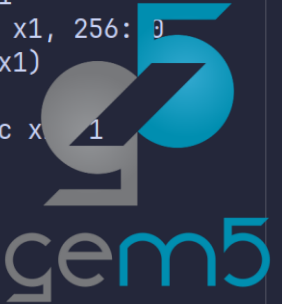
■ Comments

Gem5

- Gem5 is an open source system-level and microarchitectural simulator
- Supports various ISAs, including ARM, RISC-V and x86
- Can simulate a full system and offers fine-grained control over all components, including cache system, branch prediction and CPU functional units

```
gem5 version 22.1.0.0
gem5 compiled Aug 27 2025 11:38:55
gem5 started Sep 16 2025 11:44:37
gem5 executing on bernoulli, pid 3456889
command line: ./build/RISCV/gem5.debug --debug-flags=MinorGUI gem5_config.py --cpu-type Min
orCPU --caches --l1d_size 8388608 --l1i_size 8388608 --cacheline_size 512 --cpu-clock 1MHz
--sys-clock 10GHz -c test_progs/last/last

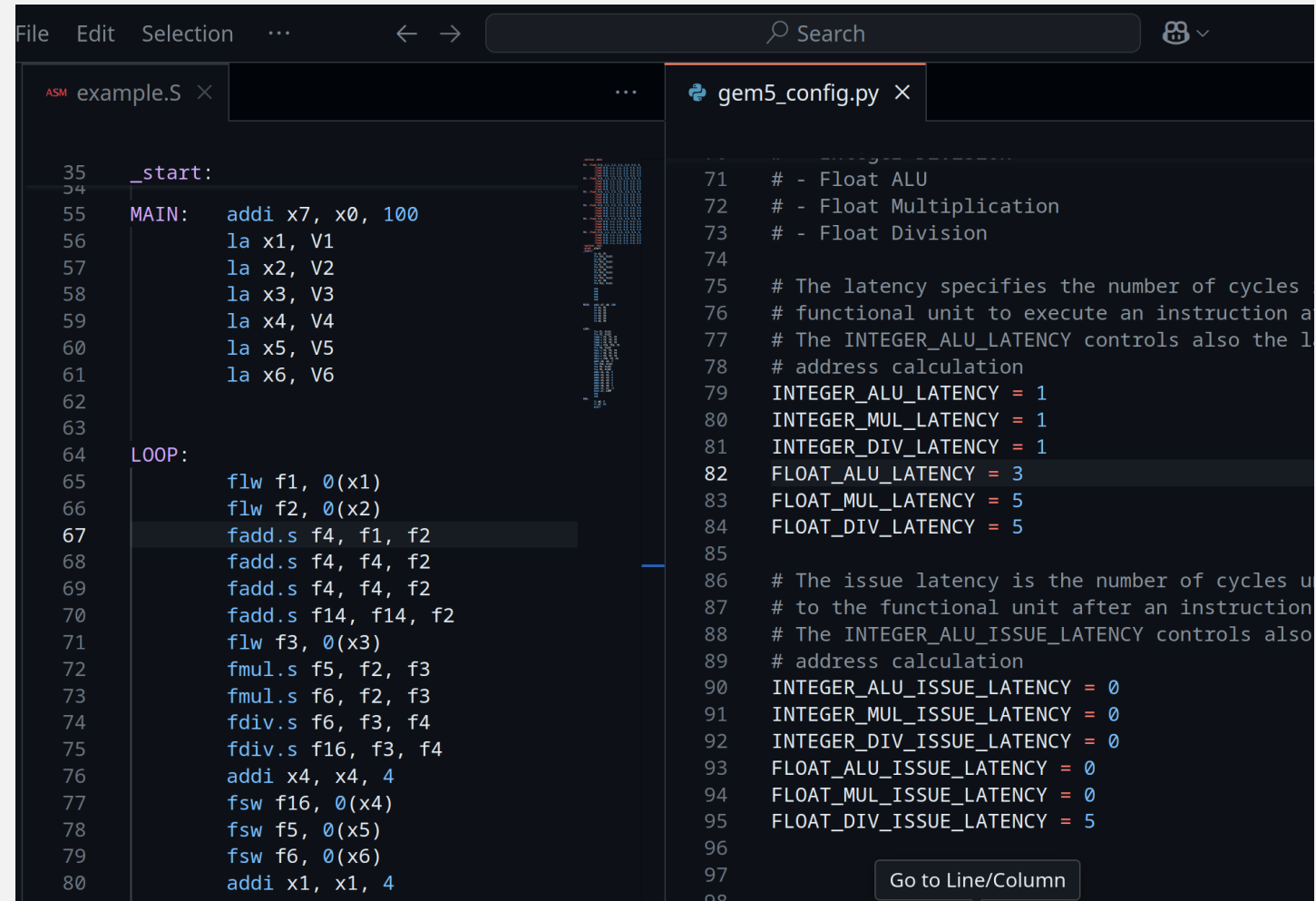
**** REAL SIMULATION ****
build/RISCV/sim/simulate.cc:192: info: Entering event queue @ 0. Starting simulation...
1000000: system.cpu.fetch1: Log4GUI: fetch1: 1000000: 0: 10074: <assembly>
2000000: system.cpu.fetch1: Log4GUI: fetch1: 2000000: 1: 10074: <assembly>
3000000: system.cpu.fetch1: Log4GUI: fetch1: 3000000: 1: 10074: <assembly>
4000000: system.cpu.fetch1: Log4GUI: fetch1: 4000000: 1: 10074: <assembly>
5000000: system.cpu.decode: Log4GUI: decode: 5000000: 0: 10074: auipc x1, 1
5000000: system.cpu.fetch1: Log4GUI: fetch1: 5000000: 0: 10078: <assembly>
6000000: system.cpu.execute: Log4GUI: execute: 6000000: 0: 10074: auipc x1, 1: 0
6000000: system.cpu.decode: Log4GUI: decode: 6000000: 0: 10078: addi x1, x1, 256
6000000: system.cpu.fetch1: Log4GUI: fetch1: 6000000: 0: 1007c: <assembly>
7000000: system.cpu.memory: Log4GUI: memory: 7000000: 0: 10074: auipc x1, 1
7000000: system.cpu.execute: Log4GUI: execute: 7000000: 0: 10078: addi x1, x1, 256: 0
7000000: system.cpu.decode: Log4GUI: decode: 7000000: 0: 1007c: flw f9, 0(x1)
7000000: system.cpu.fetch1: Log4GUI: fetch1: 7000000: 0: 10080: <assembly>
8000000: system.cpu.writeback: Log4GUI: writeback: 8000000: 0: 10074: auipc x1, 1
8000000: system.cpu.writeback: REGISTERS
8000000: global: x0=0x00000000
8000000: global: x1=0x00011074
8000000: global: x2=0x7fffff88
8000000: global: x3=0x00000000
```



Gem5 - 2

System definition and simulation

- Python scripts
 - Define components of the system
 - Including microarchitectural details, e.g. clock frequency, cache latencies, CPU functional units, ...
 - Configure and automate simulations
 - Collect statistics
- Executable code
 - Mainstream compilers (gcc, clang) can be used to build programs for simulation



The screenshot shows a code editor with two tabs: 'example.S' and 'gem5_config.py'. The 'example.S' tab displays assembly code for a RISC-V program. It starts with a '_start:' label, followed by a 'MAIN:' section where registers x1 through x6 are loaded with values from memory. Then, a 'LOOP:' section follows, containing a series of floating-point operations: loading f1 and f2, adding f4 to f1 and f2, multiplying f5 and f6 by f2 and f3, dividing f6 by f3, and finally adding 4 to x1. The 'gem5_config.py' tab shows configuration parameters for the simulation, including latencies for integer and floating-point ALU, multiplication, and division, as well as issue latencies. Comments explain that these values represent the number of cycles for a functional unit to execute an instruction or the number of cycles after an instruction to the functional unit.

```
File Edit Selection ... Search
ASM example.S x
35 _start:
36
55 MAIN: addi x7, x0, 100
56      la x1, V1
57      la x2, V2
58      la x3, V3
59      la x4, V4
60      la x5, V5
61      la x6, V6
62
63
64 LOOP:
65      flw f1, 0(x1)
66      flw f2, 0(x2)
67      fadd.s f4, f1, f2
68      fadd.s f4, f4, f2
69      fadd.s f4, f4, f2
70      fadd.s f14, f14, f2
71      flw f3, 0(x3)
72      fmul.s f5, f2, f3
73      fmul.s f6, f2, f3
74      fdiv.s f6, f3, f4
75      fdiv.s f16, f3, f4
76      addi x4, x4, 4
77      fsw f16, 0(x4)
78      fsw f5, 0(x5)
79      fsw f6, 0(x6)
80      addi x1, x1, 4

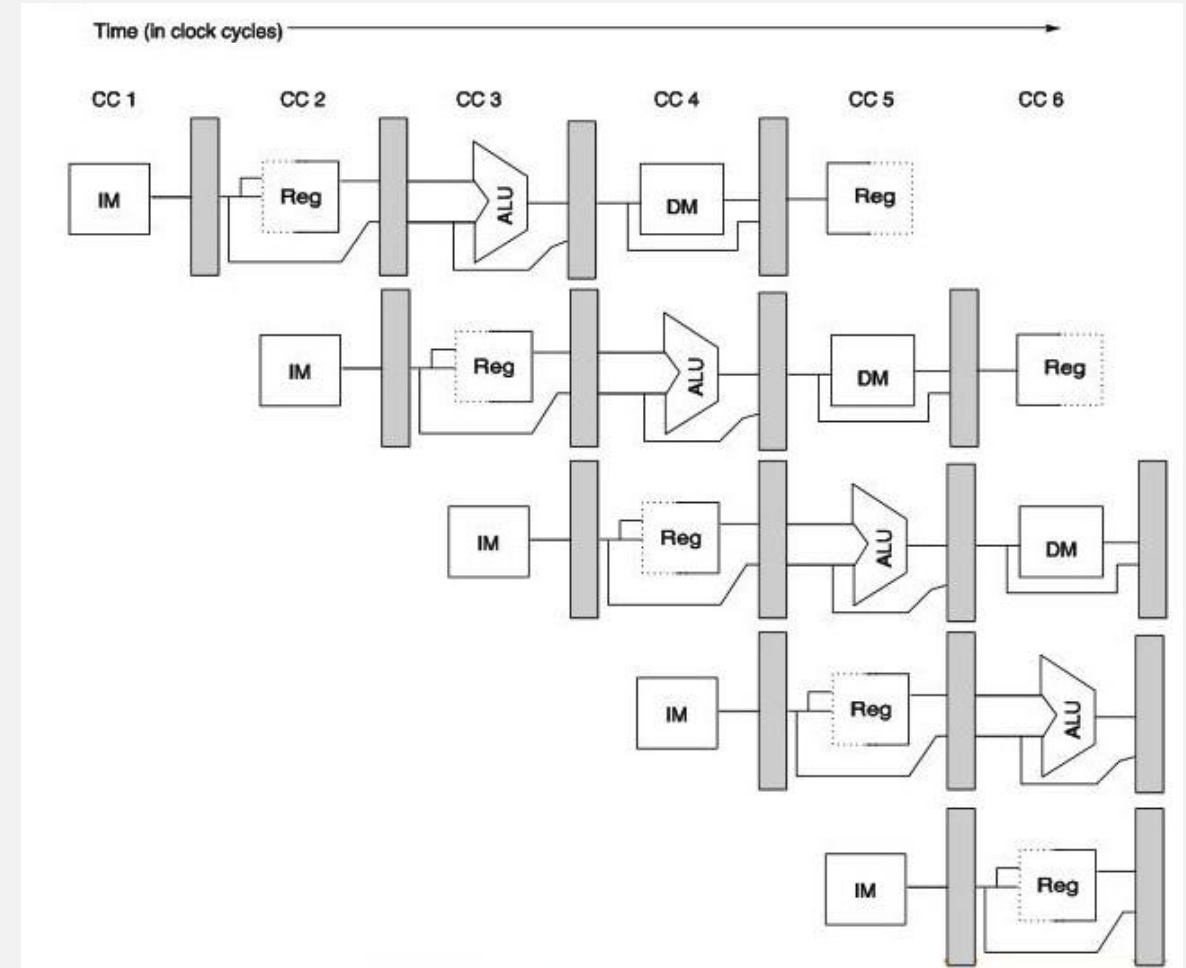
gem5_config.py x
71 # - Float ALU
72 # - Float Multiplication
73 # - Float Division
74
75 # The latency specifies the number of cycles
76 # functional unit to execute an instruction a
77 # The INTEGER_ALU_LATENCY controls also the l
78 # address calculation
79 INTEGER_ALU_LATENCY = 1
80 INTEGER_MUL_LATENCY = 1
81 INTEGER_DIV_LATENCY = 1
82 FLOAT_ALU_LATENCY = 3
83 FLOAT_MUL_LATENCY = 5
84 FLOAT_DIV_LATENCY = 5
85
86 # The issue latency is the number of cycles u
87 # to the functional unit after an instruction
88 # The INTEGER_ALU_ISSUE_LATENCY controls also
89 # address calculation
90 INTEGER_ALU_ISSUE_LATENCY = 0
91 INTEGER_MUL_ISSUE_LATENCY = 0
92 INTEGER_DIV_ISSUE_LATENCY = 0
93 FLOAT_ALU_ISSUE_LATENCY = 0
94 FLOAT_MUL_ISSUE_LATENCY = 0
95 FLOAT_DIV_ISSUE_LATENCY = 5
96
97 Go to Line/Column
```

Gem5 - 3

RISC-V 5-stage pipeline

- Custom CPU architecture for gem5 developed specifically for this course
- 5-stage pipeline: Fetch, Decode, Execute, Memory, Writeback
- Accurately mirrors the pipeline described in Hennessy-Patterson

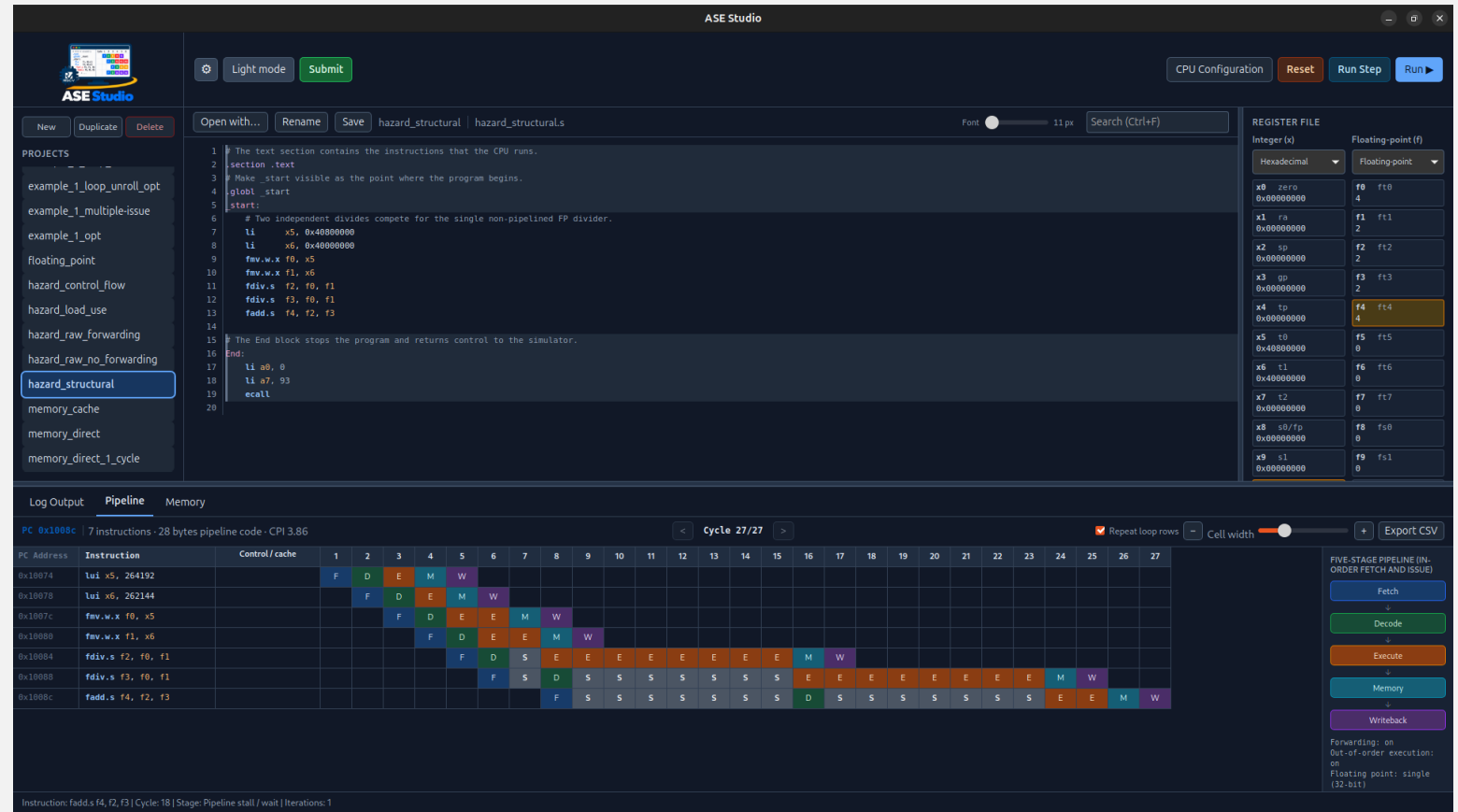
Mismatch reports are welcome!



ASE Studio V1.0.0 – 4

ASE Studio is an Interactive IDE based on GEM5 and keeps the complete experiment:

- Write RISC-V assembly
- Compilation and Simulate
- Inspect the pipeline



ASE Studio V1.0.0 – 5

Projects

Register File

Results

ASE Studio

New Duplicate Delete

PROJECTS

example_1_loop_unroll_opt

example_1_multiple-issue

example_1_opt

Floating-point

hazard_control_flow

hazard_load_use

hazard_raw_forwarding

hazard_raw_no_forwarding

hazard_structural

memory_cache

memory_direct

memory_direct_1_cycle

Light mode Submit

Assembly Editor

CPU Configuration Reset Run Step Run

Open with... Rename Save hazard_structural hazard_structurals

Font 11 px Search (Ctrl+F)

```
1 The text section contains the instructions that the CPU runs.
2 section .text
3 # Make_start visible as the point where the program begins.
4 globl _start
5 _start:
6 # Two independent divides compete for the single non-pipelined FP divider.
7 li x5, 0x40000000
8 li x6, 0x40000000
9 fmv.w.x f0, x5
10 fmv.w.x f1, x6
11 fdiv.s f2, f0, f1
12 fdiv.s f3, f0, f1
13 fadd.s f4, f2, f3
14
15 # The End block stops the program and returns control to the simulator.
16 End:
17 li a0, 0
18 li a7, 93
19 ecall
20
```

REGISTER FILE

Integer (x) Floating-point (f)

Hexadecimal Floating-point

x0 zero 0x00000000 f0 ft0 4

x1 ra 0x00000000 f1 ft1 2

x2 sp 0x00000000 f2 ft2 2

x3 gp 0x00000000 f3 ft3 2

x4 tp 0x00000000 f4 ft4 4

x5 t0 0x40000000 f5 ft5 0

x6 t1 0x40000000 f6 ft6 0

x7 t2 0x00000000 f7 ft7 0

x8 s0/fp 0x00000000 f8 fs0 0

x9 s1 0x00000000 f9 fs1 0

Log Output Pipeline Memory

PC 0x1008c | 7 instructions - 28 bytes pipeline code - CPI 3.86

< Cycle 27/27 > Repeat loop rows Cell width Export CSV

PC Address	Instruction	Control/cache	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
0x10074	lui x5, 264192		F	D	E	M	W																						
0x10078	lui x6, 262144			F	D	E	M	W																					
0x1007c	fmv.w.x f0, x5				F	D	E	E	M	W																			
0x10080	fmv.w.x f1, x6					F	D	E	E	M	W																		
0x10084	fdiv.s f2, f0, f1					F	D	S	E	E	E	E	E	E	E	E	E	M	W										
0x10088	fdiv.s f3, f0, f1						F	S	D	S	S	S	S	S	S	S	S	E	E	E	E	E	E	E	M	W			
0x1008c	fadd.s f4, f2, f3							F	S	S	S	S	S	S	S	S	S	D	S	S	S	S	S	S	E	E	M	W	

FIVE-STAGE PIPELINE (IN-ORDER FETCH AND ISSUE)

Fetch

Decode

Execute

Memory

Writeback

Forwarding: on

Out-of-order execution: on

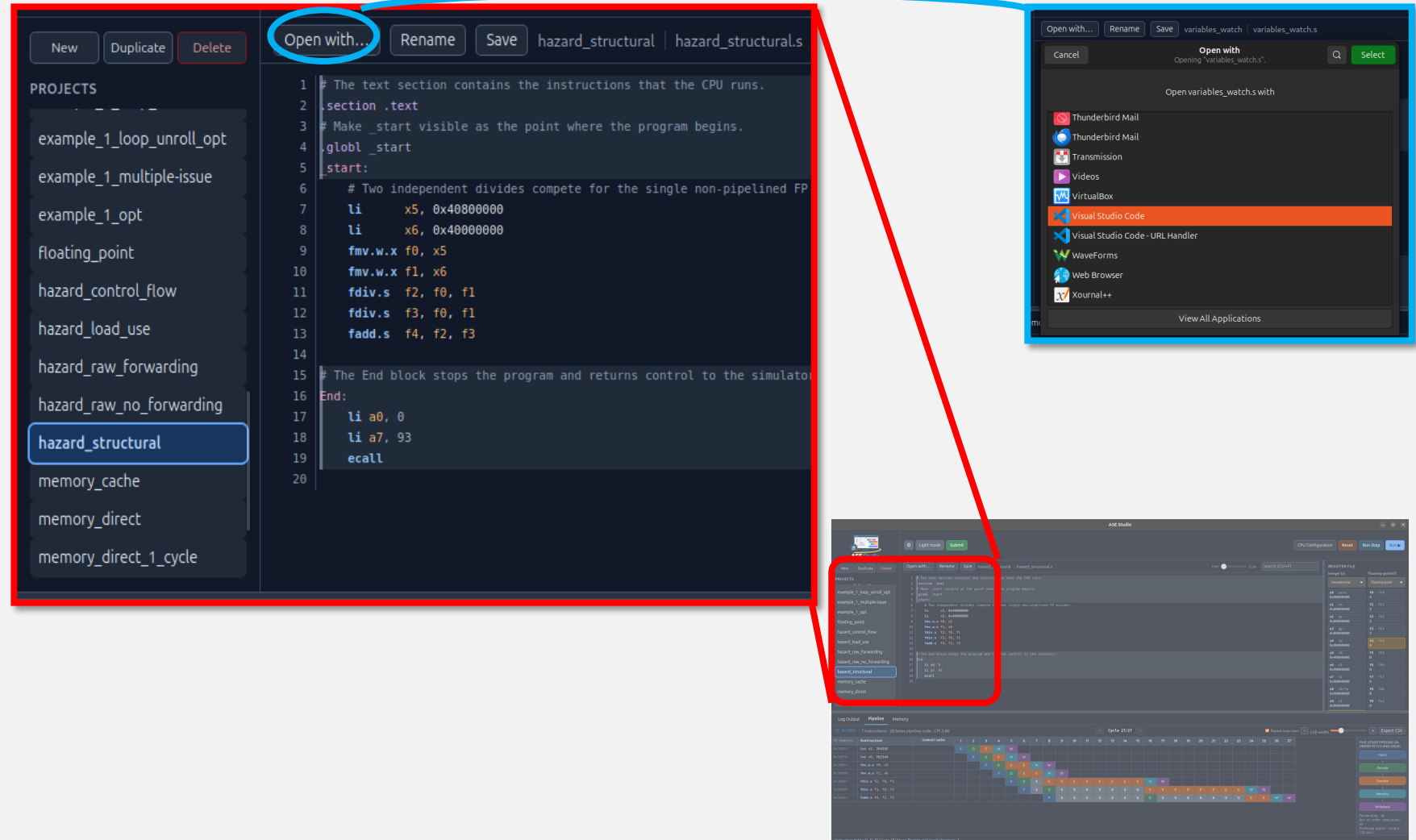
Floating-point: single (32-bit)

Instruction: fadd.s f4, f2, f3 | Cycle: 18 | Stage: Pipeline stall / wait | Iterations: 1

ASE Studio V1.0.0 – 6

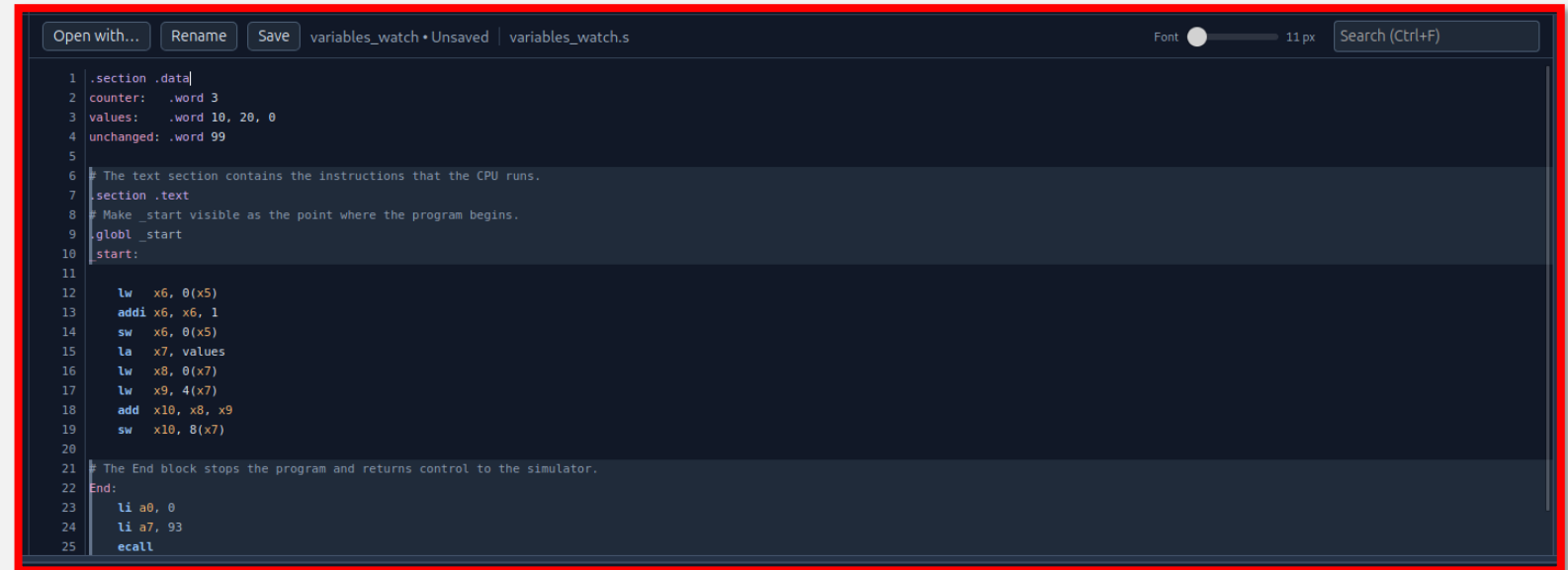
Edit Projects

- Create New project
- Duplicate a selected project
- Delete a project
- Rename/Save of current project
- Optionally open your assembly code with your preferred editor



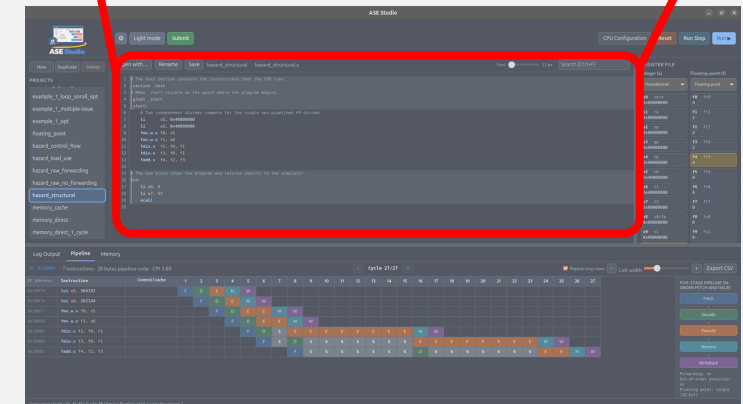
ASE Studio V1.0.0 – 7

- The protected section (highlighted in gray) are necessary for all projects and needed by simulator
- Font size is changeable
- The search box highlights the matched texts



The screenshot shows the ASE Studio code editor with a dark theme. At the top, there is a toolbar with buttons for 'Open with...', 'Rename', and 'Save'. To the right of these buttons is a search bar labeled 'Search (Ctrl+F)'. The code is written in assembly language and is divided into sections: a data section (lines 1-4), a text section (lines 6-10), and an end block (lines 21-25). The text section is highlighted in gray. The code includes instructions like `lw`, `addi`, `sw`, `la`, `add`, and `ecall`.

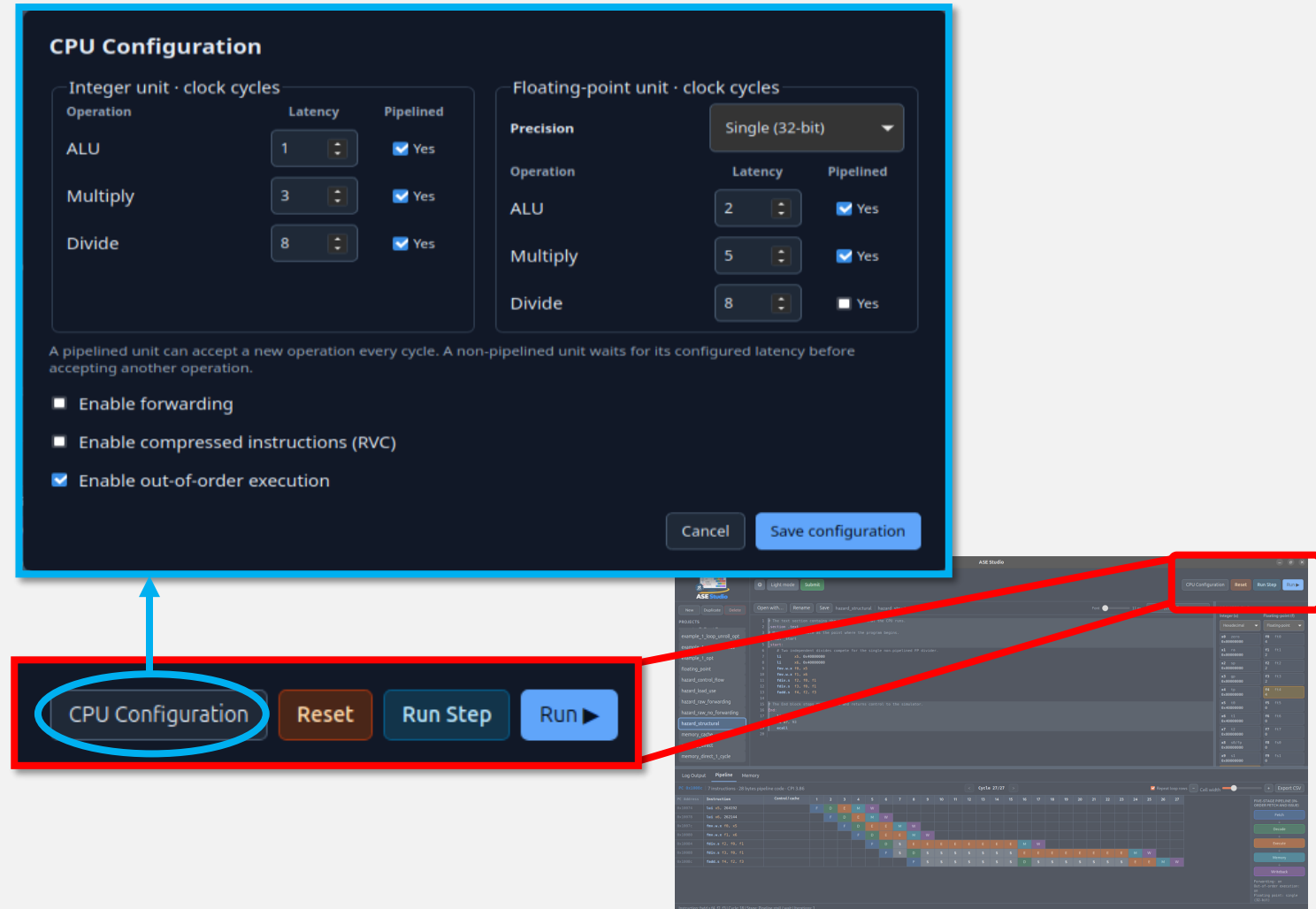
```
1 .section .data
2 counter: .word 3
3 values: .word 10, 20, 0
4 unchanged: .word 99
5
6 # The text section contains the instructions that the CPU runs.
7 .section .text
8 # Make_start visible as the point where the program begins.
9 .globl _start
10 _start:
11
12     lw x6, 0(x5)
13     addi x6, x6, 1
14     sw x6, 0(x5)
15     la x7, values
16     lw x8, 0(x7)
17     lw x9, 4(x7)
18     add x10, x8, x9
19     sw x10, 8(x7)
20
21 # The End block stops the program and returns control to the simulator.
22 End:
23     li a0, 0
24     li a7, 93
25     ecall
```



ASE Studio V1.0.0 – 8

CPU Configuration

- Different functional units can be defined. Integer and Floating-Point (FP) units have separate latencies
- Single or double precision
- FP units can be defined as Pipelined or not.
- Forwarding lets dependent instructions consume results before register-file writeback



ASE Studio V1.0.0 – 9

RUN your code

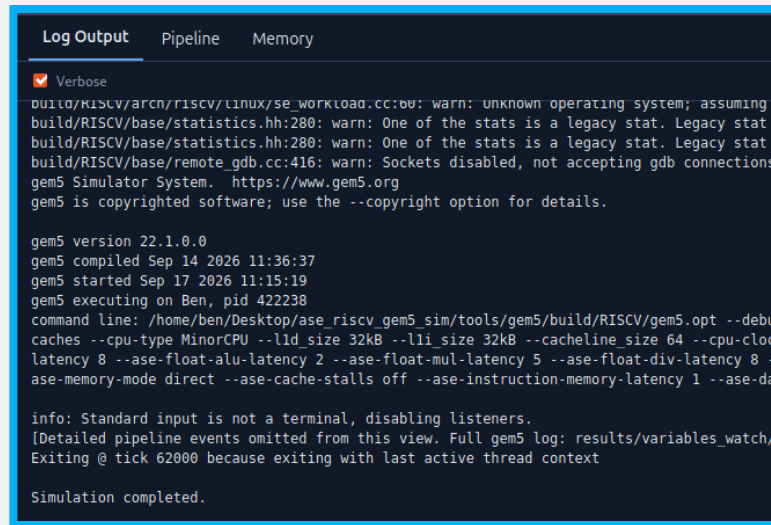
- **Run** saves, builds, simulates the complete program, and loads the final pipeline
- **Run Step** rebuilds and restarts, then stops the display at cycle 1
- **Next Cycle** advances one cycle; Restart Step rebuilds when the end is reached
- **Reset** clears pipeline/playback state



ASE Studio V1.0.0 – 9

Simulation Results

- Log output reports compile, build and simulation outcome
- Verbose will show the total logs of all steps



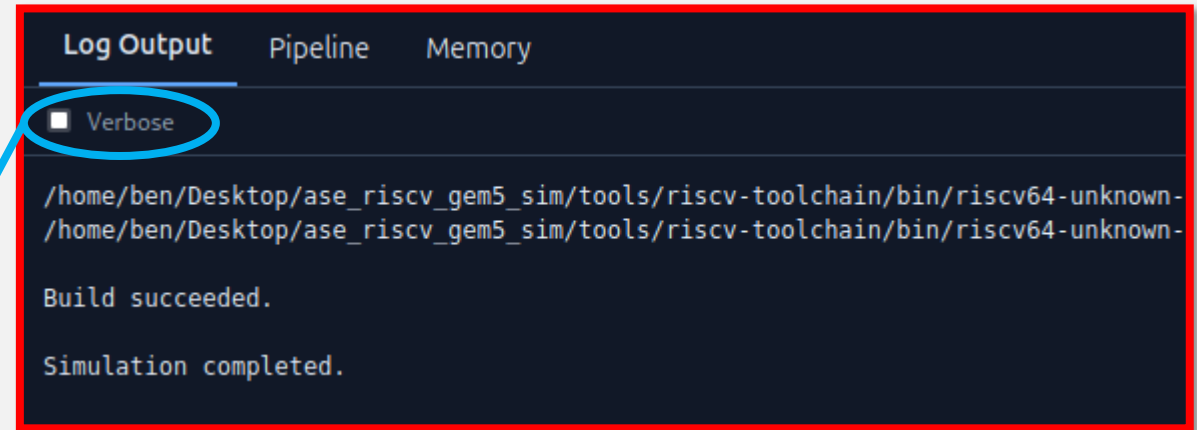
The screenshot shows the ASE Studio interface with the 'Log Output' tab selected. The 'Verbose' checkbox is checked. The log text includes:

```
outio/riscv/arcn/riscv/linux/se workload.cc:bw: warn: unknown operating system; assuming
build/RISCV/base/statistics.hh:280: warn: One of the stats is a legacy stat. Legacy stat
build/RISCV/base/statistics.hh:280: warn: One of the stats is a legacy stat. Legacy stat
build/RISCV/base/remote_gdb.cc:416: warn: Sockets disabled, not accepting gdb connections
gem5 Simulator System. https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 22.1.0.0
gem5 compiled Sep 14 2026 11:36:37
gem5 started Sep 17 2026 11:15:19
gem5 executing on Ben, pid 422238
command line: /home/ben/Desktop/ase_riscv_gem5_sim/tools/gem5/build/RISCV/gem5.opt --debu
caches --cpu-type MinorCPU --l1d_size 32kB --l1i_size 32kB --cacheline_size 64 --cpu-cloc
latency 8 --ase-float-alu-latency 2 --ase-float-mul-latency 5 --ase-float-div-latency 8 --
ase-memory-mode direct --ase-cache-stalls off --ase-instruction-memory-latency 1 --ase-da

info: Standard input is not a terminal, disabling listeners.
[Detailed pipeline events omitted from this view. Full gem5 log: results/variables_watch/
Exiting @ tick 62000 because exiting with last active thread context

Simulation completed.
```

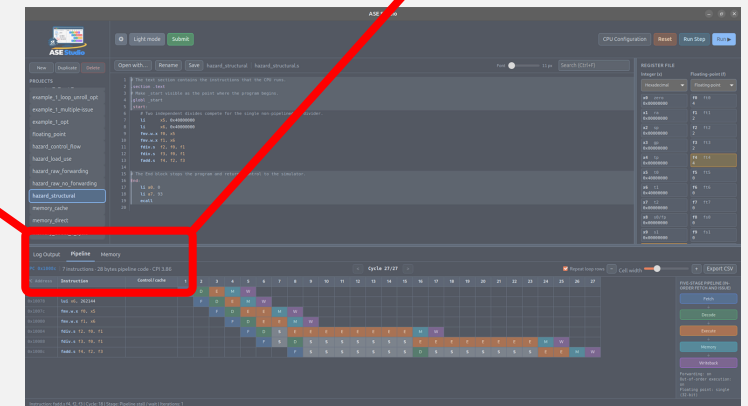


The screenshot shows the ASE Studio interface with the 'Log Output' tab selected. The 'Verbose' checkbox is unchecked. The log text includes:

```
/home/ben/Desktop/ase_riscv_gem5_sim/tools/riscv-toolchain/bin/riscv64-unknown-
/home/ben/Desktop/ase_riscv_gem5_sim/tools/riscv-toolchain/bin/riscv64-unknown-

Build succeeded.

Simulation completed.
```



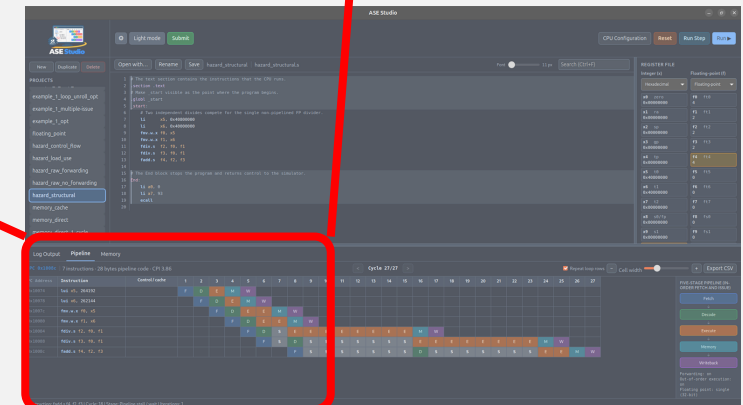
ASE Studio V1.0.0 – 10

Pipeline evolution

- Each row is an executed instruction; each numbered column is one clock cycle
- PC Address identifies the compiled instruction
- In the current version, Control/cache column reports jumps addresses
- Repeated stage letters mean a multi-cycle stage; S means a pipeline stall.

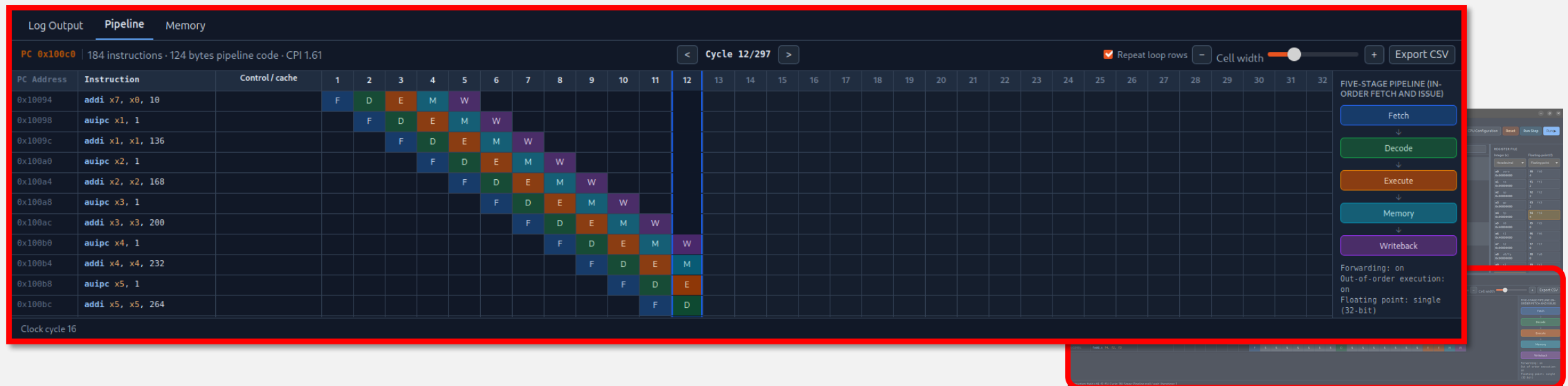
Log Output Pipeline Memory										
PC 0x10088 12 instructions · 24 bytes pipeline code · CPI 1.92										
PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	
0x10074	addi x5, x0, 0		F	D	E	M	W			
0x10078	addi x6, x0, 4			F	D	E	M	W		
0x1007c	addi x5, x5, 1				F	D	E	M	W	
0x10080	blt x5, x6, -4	~ 0x1007c				F	S	D	E	
0x10084	addi x7, x5, 10							F		
0x1007c	addi x5, x5, 1									F

F	Fetch
D	Decode
E	Execute
M	Memory
W	Writeback
S	Stall



ASE Studio V1.0.0 – 11

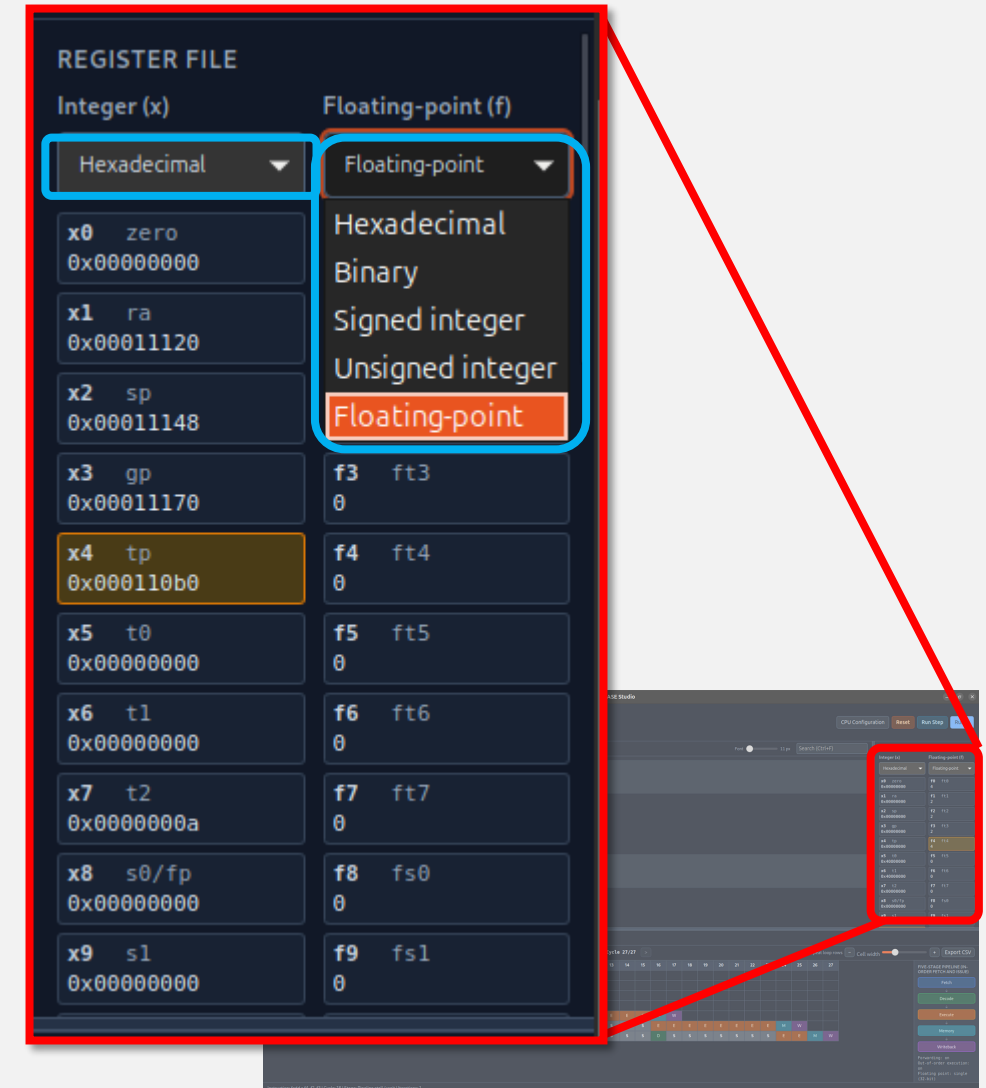
- The stage diagram on the right summarizes the active processor configuration
- Use -, +, or Cell width to fit more cycles or make cells easier to read
- Drag empty table space with the left mouse button
- Right-click a cell to highlight its row and cycle and jump to the matching editor line
- In Run Step mode, < and > move backward or forward



ASE Studio V1.0.0 – 12

Register Files

- Integer registers show x numbers and ABI aliases; floating registers show f numbers and ABI aliases
- Choose formats independently: hexadecimal, binary, signed, unsigned, or floating-point
- During Run Step, only registers changed in the current cycle are highlighted
- Floating registers use 32 or 64 bits according to the selected precision



ASE Studio V1.0.0 – 13

Memory

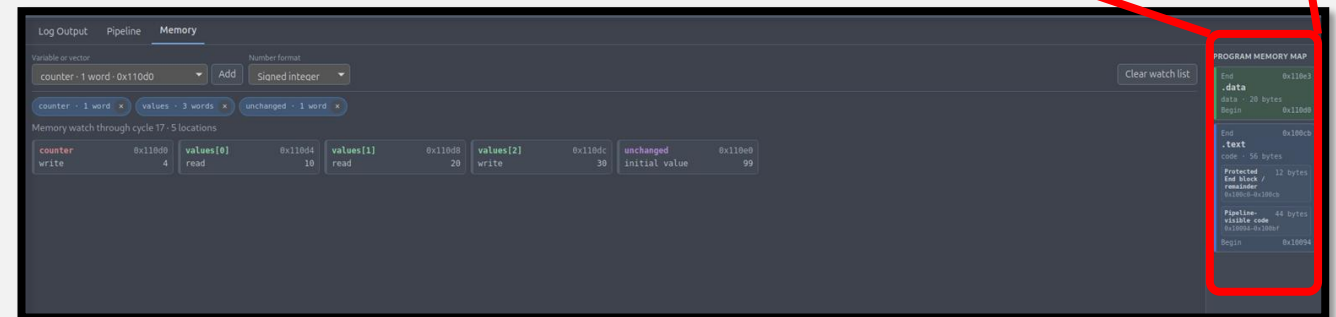
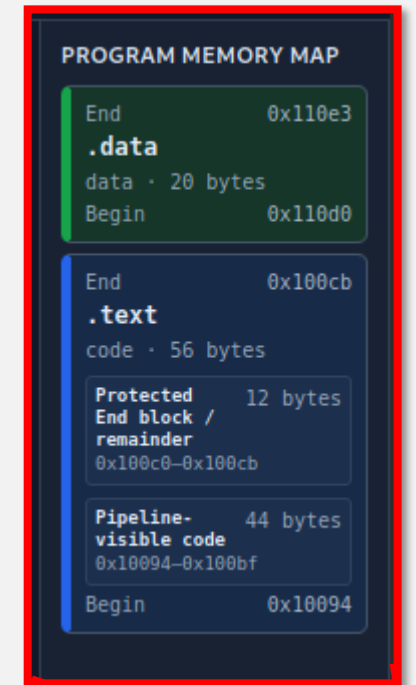
- The Memory tab shows **.data** section from the compiled program
- Select a variable or vector and choose Add.
- you can remove each individually or clear the list.
- Vector elements use names such as V1[0], V1[1], etc.
- Run Step highlights locations read or written in the current cycle.

Variable	Operation	Address	Value
counter	write	0x110d0	4
values[0]	read	0x110d4	10
values[1]	read	0x110d8	20
values[2]	write	0x110dc	30
unchanged	initial value	0x110e0	99

ASE Studio V1.0.0 – 14

Memory Map

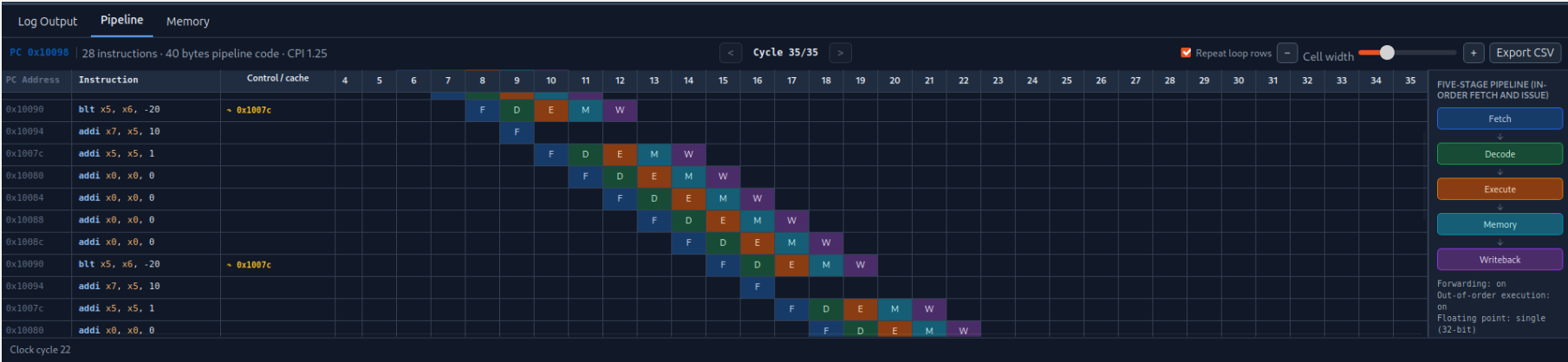
- The diagram orders allocated the Code and DATA sections described in the ELF file by address, showing the *Begin* and *End* addresses
- The *Text* section is separated into pipeline-visible code and the protected exit block
- Data sections show compiled values to store for variables and vectors.



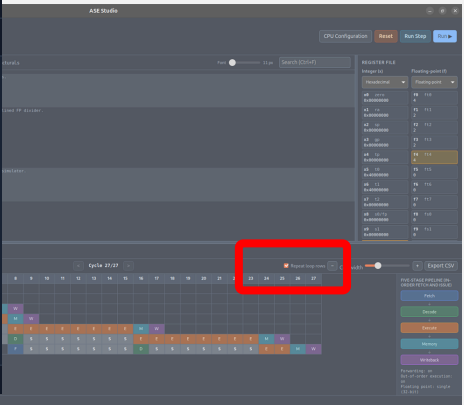
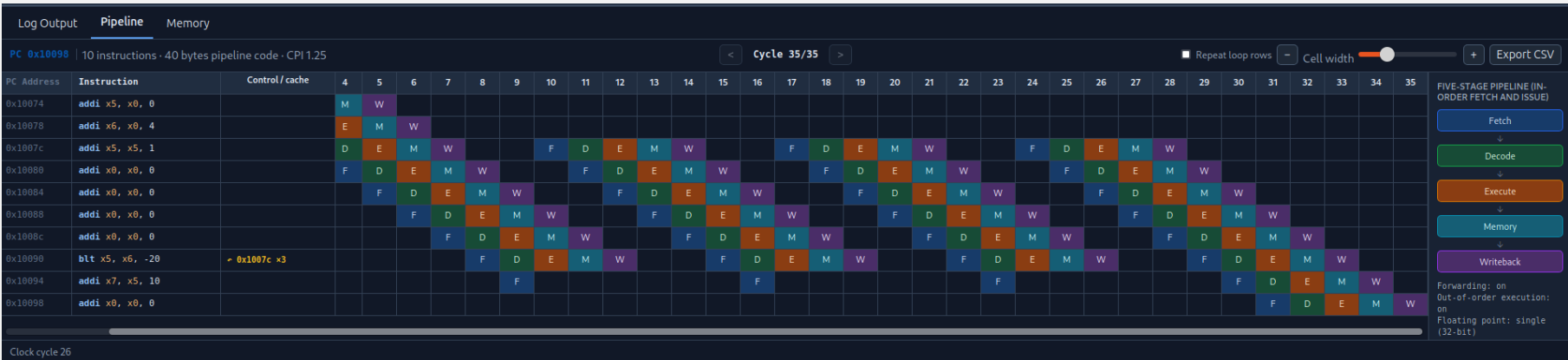
ASE Studio V1.0.0 – 15

Hints

- Repeat loop rows shows each dynamic iteration on a new row.



- Disable it to merge all occurrences of each static instruction into one compact row



ASE Studio V1.0.0 – 16

Sharing the results

- Export CSV saves the currently visible rows and cycles.
- During Run Step, export contains cycles only up to the current playback position.
- For traces beyond the display limit, ASE Studio automatically creates the complete CSV and reports its path.

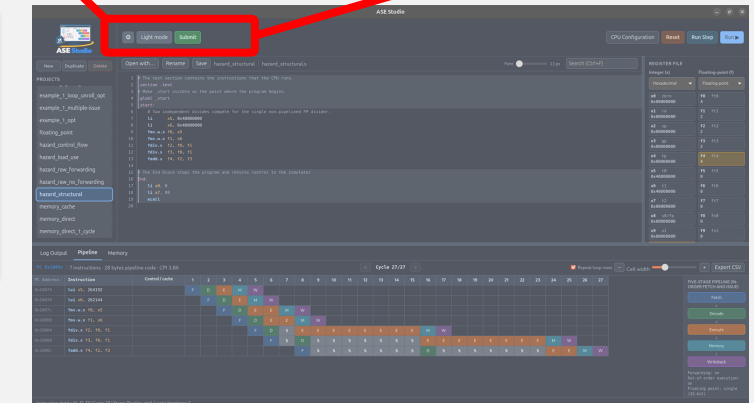
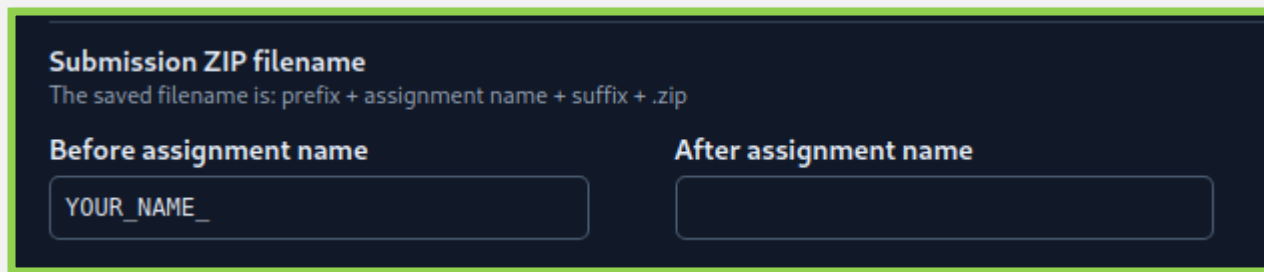
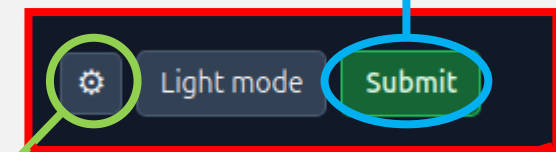
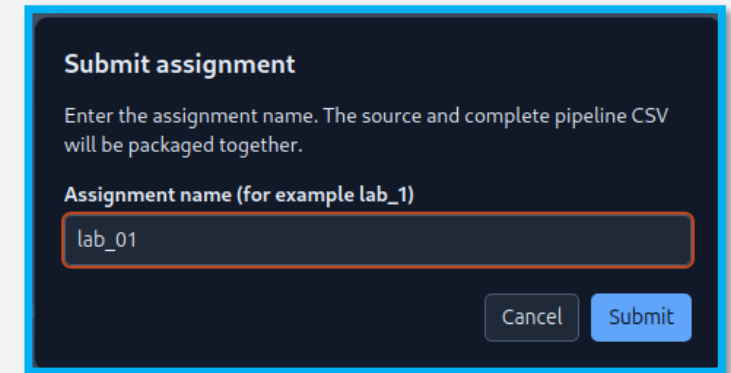
The screenshot shows the ASE Studio V1.0.0 interface. The Pipeline tab is active, displaying a table of instructions and their execution status across 16 cycles. The Memory tab is also visible, displaying a table of memory addresses and their values across 16 cycles. The Export CSV button is highlighted in the bottom right corner.

PC Address	Instruction	Control / cache	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0x10094	auipc x5, 1		F	D	E	M	W											
0x10098	addi x5, x5, 60			F	D	E	M	W										
0x1009c	lw x6, 0(x5)				F	D	E	M	W									
0x100a0	addi x6, x6, 1					F	D	S	E	M	W							
0x100a4	sw x6, 0(x5)						F	S	D	E	M	W						
0x100a8	auipc x7, 1							F	D	E	M	W						
0x100ac	addi x7, x7, 44								F	D	E	M	W					
0x100b0	lw x8, 0(x7)									F	D	E	M	W				
0x100b4	lw x9, 4(x7)										F	D	E	M	W			
0x100b8	add x10, x8, x9											F	D	S	E	M	W	
0x100bc	sw x10, 8(x7)												F	S	D	E	M	

	A	B	C	D	E	F	G	H	I	J	
1	Pipeline summary	Value									
2	Total cycles	17									
3	Executed instructions	11									
4	Pipeline code size (bytes)	44									
5	Branch prediction misses	0									
6	Stalls	4									
7	CPI	1.545									
8											
9	PC Address	Instruction	Control flow / cache	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
10	0x10094	auipc x5, 1		F	D	E	M	W			
11	0x10098	addi x5, x5, 60			F	D	E	M	W		
12	0x1009c	lw x6, 0(x5)				F	D	E	M	W	
13	0x100a0	addi x6, x6, 1					F	D	S	E	M
14	0x100a4	sw x6, 0(x5)						F	S	D	E
15	0x100a8	auipc x7, 1								F	D
16	0x100ac	addi x7, x7, 44									F
17	0x100b0	lw x8, 0(x7)									
18	0x100b4	lw x9, 4(x7)									
19	0x100b8	add x10, x8, x9									
20	0x100bc	sw x10, 8(x7)									
21											
22											
23											
24											

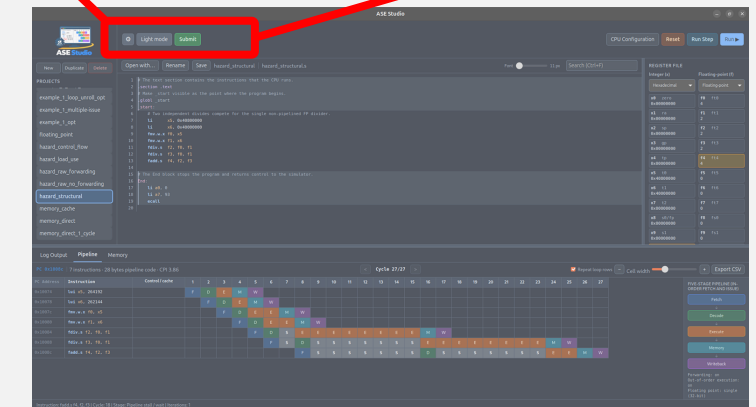
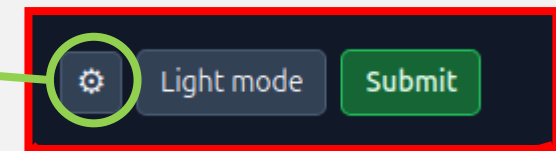
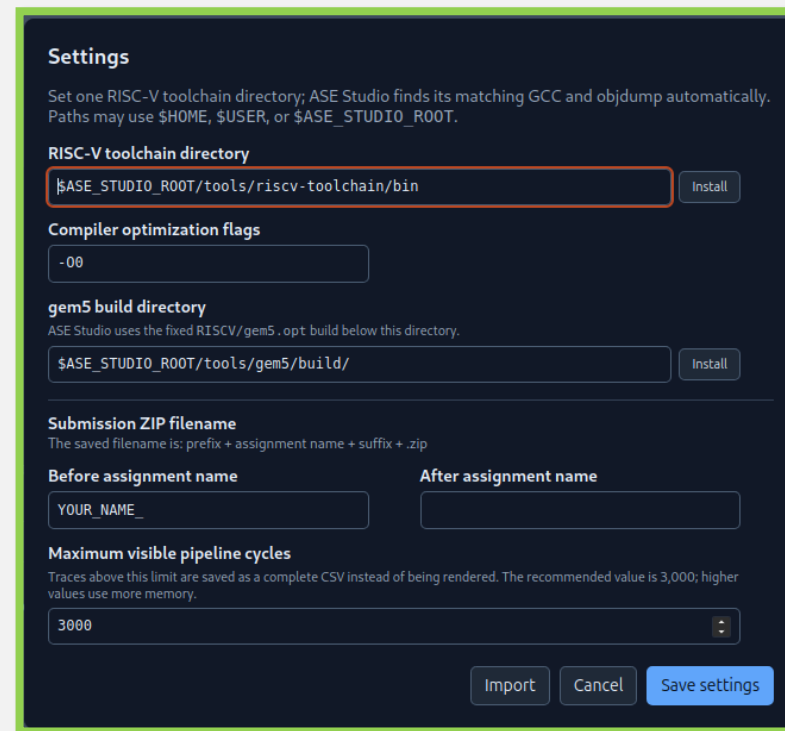
ASE Studio V1.0.0 – 17

- **Submit** asks for the assignment name, then builds and simulates the current project
- Submission succeeds only when the assembly compiles and simulation completes
- The ZIP file contains the assembly source, and the executed pipeline CSV.
- **Settings** controls the filename prefix and suffix required by the course.



ASE Studio V1.0.0 – 18

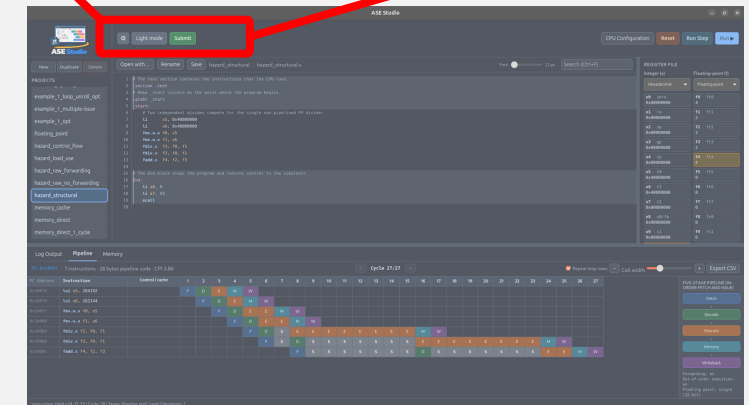
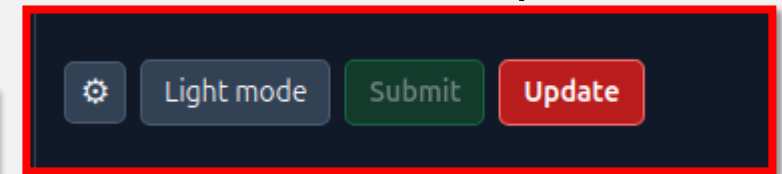
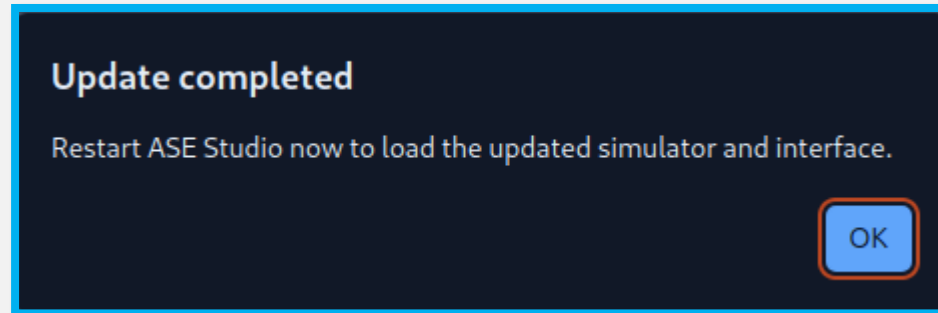
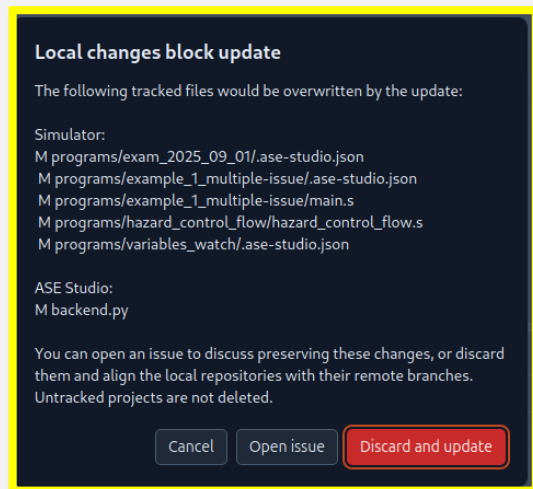
- Settings validates the RISC-V toolchain directory and fixed RISC-V/gem5.opt build
- Compiler optimization flags affect generated instructions and therefore the pipeline
- Maximum visible cycles protects the browser from very large tables.



ASE Studio V1.0.0 – 19

ASE Studio updates

- **Update** appears only when the simulator, ASE Studio, or configured gem5 repository has a newer commit.
- If tracked local changes conflict, choose whether to open an issue or discard and align with the official repository.
- After a successful update, restart ASE Studio; gem5 is rebuilt automatically when needed.



A first example

$$C = A + B$$

```
.section .data
Val_A: .word 10
Val_B: .word 20
Val_C: .word 0
.section .text
.globl _start
_start:
    la x1, Val_A
    lw x1, 0(x1)
    la x2, Val_B
    lw x2, 0(x2)
    add x3, x2, x1
    la x4, Val_C
    sw x3, 0(x4)
```

RISC-V PROGRAM

```
.data
Val_A: dw 10
Val_B: dw 20
Val_C: dw 0

...
Main:
    mov AX, Val_A
    add AX, Val_B
    mov Val_C, AX
```

8086 PROGRAM

A first example (I)

$C = A + B$

```
.section .data
Val_A: .word 10
Val_B: .word 20
Val_C: .word 0
.section .text
.globl _start
_start:
    la x1, Val_A
    lw x1, 0(x1)
    la x2, Val_B
    lw x2, 0(x2)
    add x3, x2, x1
    la x4, Val_C
    sw x3, 0(x4)
```

RISC-V PROGRAM

Code Analysis	
# instructions	7 → 10
Code size [bytes]	40
Execution time [C.C.]	15

A first example (II)

$$C = A + B$$

Code Analysis	
# Instructions	3
Code size [bytes]	8
Execution time [C.C.]	33

```
.data
Val_A: dw 10
Val_B: dw 20
Val_C: dw 0

...
Main:
    mov AX, Val_A
    add AX, Val_B
    mov Val_C, AX
```

8086 PROGRAM

A 2nd example

```
/* ***** */
/* Sum of two vectors */
/* ***** */

#include <stdio.h>

const int V1[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
const int V2[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
int result[10];

int main() {
    for (int i = 0; i < 10; i++) {
        result[i] = V1[i] + V2[i];
    }
}
```

A 2nd example

```
***** RISC-V INITIAL PROGRAM *****
#-----
# AddTwoVectors.s
# this program adds two vectors and stores
# the result in another vector
#-----

        .section .data
V1:      .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
V2:      .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
result:  .space 40
        .section .text
# This prologue ensures that vectors are in
# the cache when we need them, otherwise
# the pipeline will stall at first access
        .globl _start
_start:
        nop
main:
        la x1, V1
        la x2, V2
        la x3, result
        addi x4, x0, 10 # iteration counter
```

Since we don't have a main, the `_start` symbol indicates where the beginning of the program is. **Always use the `_start` label, not `start`, not `START`, not `main`!**

cycle:

```
lw x5, 0(x1)
lw x6, 0(x2)
add x7, x5, x6
sw x7, 0(x3)
addi x1, x1, 4
addi x2, x2, 4
addi x3, x3, 4
addi x4, x4, -1
bnez x4, cycle
nop
```

END_of_PROGRAM:

```
li a0, 0 # Syscall parameter
li a7, 93 # exit() syscall number
ecall
```

Use the `exit()` syscall to stop the gem5 simulation

References

- <https://github.com/cad-polito-it/gem5>
- <https://github.com/cad-polito-it/ase-studio>
- https://github.com/cad-polito-it/ase_riscv_gem5_sim
- https://www.gem5.org/documentation/general_docs/building